

IO SKY

AUDIT LOG INTEGRITY & GOVERNANCE REQUIREMENTS

Developer Handoff

Version 1.0 - Functional Master

Status: Functional, Security & Governance Requirements

Audience

Senior Full-Stack Developers | Backend | Platform | Security | DevOps | QA

Authoritative developer-facing functional master

1. DOCUMENT PURPOSE

This document defines the mandatory functional, security, governance, integrity, access-control, investigation and usability requirements for the IO SKY Audit Log.

The Audit Log must provide a trustworthy historical record of security-relevant, governance-relevant, privileged and material business actions performed within the IO SKY platform.

Its purpose is to answer, where applicable:

- Who performed an action?
- What happened?
- When did it happen?
- Which resource was affected?
- Which Client / organisation did it concern?
- Which Project, AI Scan or other platform object did it concern?
- Which environment did it occur in?
- What was the relevant previous state?
- What was the resulting state?
- Did the action succeed, fail or get denied?
- Was the actor a human user or a system/service actor?
- Can the integrity of the recorded evidence be trusted?

The Audit Log must remain useful even when the platform contains very large numbers of audit records.

It must therefore provide both:

1. strong integrity and accountability, and
2. practical investigation and retrieval capabilities.

The Audit Log is not merely a technical logging console.

It is a controlled security, governance and accountability record for IO SKY.

2. AUDIT LOG VERSUS NOTIFICATION CENTER

The Audit Log and Notification Center are separate platform capabilities.

Notification Center

The Notification Center answers:

What does this authorised user need to know, prepare for or act on now?

It is selective, attention-oriented and intentionally low-noise.

Audit Log

The Audit Log answers:

What actually happened, who or what performed it, when did it occur, what did it affect and what was the outcome?

The Audit Log exists for:

- accountability;
- security investigation;
- governance;
- privileged-action traceability;
- incident investigation;
- historical reconstruction;
- access investigation;
- operational evidence;
- compliance support where applicable.

An underlying action may therefore:

- create a Notification;
- create an Audit record;
- create both;
- or create neither,

depending on its purpose.

Routine audit activity must not automatically generate Notification Center noise.

3. CORE AUDIT LOG PRINCIPLE

The IO SKY Audit Log must be:

Append-only from the application perspective + tamper-evident + separately authorised + tenant-aware + environment-aware + security-minimised + searchable + filterable + investigation-ready + policy-retained.

Once an audit record has been successfully committed, normal application functionality must not permit that record to be:

- edited;
- rewritten;
- overwritten;
- individually deleted;
- silently replaced;
- retroactively corrected;
- or removed because the underlying resource was subsequently modified or deleted.

A later event must create a new audit record.

Example:

Permission Granted

followed later by:

Permission Revoked

must result in two historical records.

The earlier Permission Granted record must not be rewritten to say that permission was never granted.

4. AUDIT RECORD IMMUTABILITY

Audit records must be immutable through normal platform functionality.

This applies to:

- Client;
- Developer;
- Technical Operator;
- Admin;
- Super Admin.

Even Super Admin must not receive a normal Edit or Delete capability for individual audit records.

The Super Admin's authority over the platform must not mean that historical evidence can be rewritten through the application.

Where corrections or explanatory context are legitimately required, the original record must remain intact.

A separate subsequent record or authorised annotation mechanism may be used if technically appropriate, provided the original evidence remains preserved and the addition itself is traceable.

The development team may determine the appropriate implementation architecture.

The functional result is mandatory:

Historical audit evidence must not be silently rewritten.

5. TAMPER-EVIDENT INTEGRITY

Application-level immutability alone is insufficient.

The Audit Log must include technical integrity controls capable of detecting unauthorised:

- modification;
- deletion;
- alteration;
- replacement;
- truncation;
- manipulation;
- or other compromise

of protected audit data.

The functional requirement is:

Audit records must be append-only from the application perspective and protected by technical integrity controls capable of detecting unauthorised modification, deletion or tampering.

The exact cryptographic, database or storage architecture is not prescribed by this specification. The development team remains responsible for selecting the appropriate mechanism.

For example, the implementation may use appropriate combinations of:

- append-only storage controls;
- cryptographic verification;
- chained integrity mechanisms;
- signed records;
- immutable storage;
- protected replication;
- restricted database privileges;
- external integrity evidence;
- or other suitable technical controls.

These examples are not implementation mandates.

The development team must document:

1. the chosen integrity architecture;
2. what type of tampering it detects;
3. what it does not protect against;
4. which privileged systems or actors could technically alter the underlying data;
5. how such alteration would be detected;
6. how integrity is verified;
7. how integrity failure is surfaced;
8. how recovery/investigation is performed.

6. AUTHORITATIVE SERVER-SIDE AUDITING

Audit evidence must be generated from authoritative backend/server-side activity where technically applicable.

The system must not rely exclusively on:

- frontend JavaScript;
- UI state;
- Client-provided timestamps;
- Client-provided role information;
- Client-provided outcome information;
- or other user-controlled data

for security-critical audit evidence.

The backend must determine authoritative information such as:

- authenticated actor;
- effective authorization;
- affected resource;
- environment;
- outcome;
- authoritative timestamp.

Frontend telemetry may supplement auditing where useful, but it must not become the sole evidence source for privileged or security-sensitive actions.

7. EVENTS THAT MUST BE AUDITED

Not every click or backend process belongs in the governance Audit Log.

The following categories must be auditable where applicable.

7.1 Authentication and Account Security

Examples include:

- security-relevant login events;
- significant failed or denied authentication activity;
- MFA enrollment;
- MFA removal;
- MFA reset/recovery;
- passkey/security credential changes where applicable;
- password/security recovery actions;
- forced logout/session revocation;

- account disabling;
- account reactivation;
- significant account security changes.

Routine successful activity does not need to create unnecessary noise unless required for the approved security model.

8. ROLES, PERMISSIONS AND ACCESS CHANGES

The Audit Log must record material access-control changes, including:

- role assignment;
- role removal;
- permission granted;
- permission revoked;
- scope granted;
- scope revoked;
- privileged access granted;
- privileged access revoked;
- Security scope changes;
- environment access changes;
- resource assignment changes where access is affected;
- other significant authorization changes.

The audit record must identify:

- who initiated the change;
- who or what was affected;
- relevant previous authorization state;
- relevant resulting authorization state;
- timestamp;
- outcome.

A delegated user must not be able to hide their own access changes by altering Audit Log history.

9. SUPER ADMIN AND PRIVILEGED ACTIONS

Material Super Admin and other privileged actions must be auditable.

Examples include:

- role/permission administration;
- privileged user creation;
- privileged access changes;
- Security Center actions;
- Production-sensitive changes;
- Audit Log exports;
- sensitive platform configuration changes;
- AI Scan approval/publication actions;
- credential/security configuration actions;
- governance changes;
- environment-level privileged operations.

Super Admin actions are not exempt from auditing.

The fact that an actor possesses the highest platform role increases rather than removes the need for accountability.

10. AI SCAN AUDIT REQUIREMENTS

The AI Scan lifecycle contains several business-critical and privileged actions that must be separately traceable.

The platform must audit at minimum:

- relevant Scan creation;
- submission where material;
- reviewer assignment;
- reviewer reassignment;
- review completion;
- changes required/reprocessing decision where relevant;
- approval;

- publication attempt;
- successful publication;
- failed publication;
- relevant publication correction/supersession where supported;
- relevant privileged access to Scan workflow where required.

The Audit Log must preserve the separation between:

Review

and

Publish.

A reviewer's approval and a publisher's publication must therefore be distinguishable audit events.

11. AI SCAN APPROVAL AUDIT RECORD

When an authorised Reviewer clicks Approve, the Audit Log must be able to establish at minimum:

- Reviewer identity;
- relevant role/authorization context;
- Scan ID;
- Client/organisation;
- Scan type;
- approved report version;
- report language where relevant;
- approval timestamp;
- outcome;
- relevant lifecycle transition.

The actual sensitive findings do not need to be duplicated into the Audit Log.

The Audit Log should identify which version was approved, not unnecessarily copy the entire report.

12. AI SCAN PUBLICATION AUDIT RECORD

When an authorised Publisher clicks Publish, the Audit Log must establish at minimum:

- Publisher identity;
- authorization context;
- Scan ID;
- Client/organisation;
- approved version being published;
- report language;
- publication timestamp;
- publication outcome;
- resulting publication state.

Successful publication and failed publication must remain distinguishable.

A failed Publish attempt must not be rewritten or removed after a later successful attempt.

Example:

```
14:02 - Publish Attempt - FAILED
14:07 - Publish Attempt - SUCCESS
```

Both remain part of the historical record.

13. CLIENT / TENANT ACCESS AUDITING

Material Client/tenant access changes must be auditable.

Examples include:

- user added to Client organisation;
- user removed;
- privileged Client access changed;
- Client resource assignment changed where security-relevant;
- cross-tenant administrative action;
- material account ownership/access changes.

The Audit Log itself must respect tenant isolation.

Audit visibility must never become a mechanism for cross-Client information leakage.

14. PROJECT AUDITING

Material Project governance and accountability actions should be auditable where applicable, including:

- Project assignment;
- reassignment;
- Implementer assignment;
- Reviewer assignment;
- material privileged Project access changes;
- significant delivery-state changes;
- privileged approval/delivery actions where applicable.

Routine task edits, ordinary viewing and every minor Project interaction do not automatically require governance-level audit records.

15. SECURITY CENTER AUDITING

Material Security Center activity must be auditable.

This includes, where applicable:

- security incident creation;
- acknowledgement;
- assignment;
- escalation;
- privileged security decision;
- security configuration change;
- incident resolution;
- sensitive access change;
- other significant Security Center actions.

Audit visibility of Security Center events requires appropriate Security/Governance authorization.

An Admin or Technical Operator does not receive Security Audit access merely because of their platform role.

16. PRODUCTION-SENSITIVE TECHNICAL ACTIONS

Material Production-sensitive actions must be auditable where technically appropriate.

Examples may include:

- privileged Production configuration change;
- deployment-related privileged action;
- security configuration change;
- Production environment access change;
- database administration action with governance significance;
- infrastructure access changes;
- emergency operational actions.

This does not mean every low-level infrastructure telemetry event must be copied into the IO SKY governance Audit Log.

The objective is accountability for material privileged actions, not duplication of all infrastructure logs.

17. CREDENTIAL AND SECRET MANAGEMENT EVENTS

Credential/security lifecycle events may need to be audited.

Examples:

- credential created;
- credential rotated;
- credential revoked;
- privileged secret configuration changed;
- access to secret-management capability changed.

The Audit Log may record:

Production database credential rotated successfully

It must never record the credential value itself.

18. SYSTEM AND SERVICE ACTORS

Not every audited action is performed directly by a human.

The Audit Log must distinguish, where applicable, between:

- Human Actor
- System Actor
- Service Actor

A system-generated action must not be falsely attributed to a human user.

Where technically possible, the system/service responsible should be identifiable.

Examples:

```
Actor Type: HUMAN Actor: user_123
```

or:

```
Actor Type: SERVICE Actor: scan-processing-service
```

This supports meaningful incident reconstruction.

19. FAILED AND DENIED PRIVILEGED ACTIONS

The Audit Log must not contain only successful actions.

Security-relevant or privileged failed/denied actions must also be auditable where they provide meaningful accountability.

Examples:

- unauthorised attempt to access Security Center;
- denied Production access;
- failed privileged configuration change;
- failed AI Scan Publish;
- repeated denied privileged action;
- failed governance operation.

Not every ordinary 403 or harmless application error needs a governance-level Audit record.

The implementation must distinguish routine errors from meaningful security/accountability events.

20. MINIMUM AUDIT RECORD DATA MODEL

Each audit record must support the following fields where applicable.

Identity

- unique Audit Record ID;
- stable Action/Event Type;
- Actor ID;
- Actor Type;
- Actor Role;
- relevant effective permission/scope context.

Resource

- Resource Type;
- Resource ID;
- Client / Organisation ID;
- Project ID where relevant;
- AI Scan ID where relevant;
- other relevant parent/resource relationship.

Environment

- Development;
- Staging;
- Production;
- or another explicitly approved environment.

Time

- authoritative server timestamp.

Action

- action performed/attempted;
- relevant previous state;
- relevant resulting state.

Outcome

At minimum where applicable:

- SUCCESS
- FAILED
- DENIED

Additional technically useful outcome states may be used where necessary.

Context

Where safe and appropriate:

- correlation/request ID;
- session reference;
- source context;
- reason/justification;
- IP/network context;
- device/security context.

Context must be subject to security, privacy and minimisation requirements.

21. BEFORE / AFTER STATE

For material state-changing actions, the Audit Log should provide sufficient information to understand what changed.

Example:

Before

```
scan.publish = false
```

After

```
scan.publish = true
```

or:

Before

```
Reviewer = Developer A
```

After

```
Reviewer = Developer B
```

The Audit Log must not blindly store complete database objects.

Only the relevant safe state differences should be recorded.

Sensitive values must be redacted or omitted.

22. DATA THAT MUST NEVER BE STORED IN AUDIT RECORDS

The Audit Log must never contain raw:

- passwords;
- password reset secrets;
- MFA secrets;
- TOTP seeds;
- passkey private credential material;
- API keys;
- access tokens;
- refresh tokens;
- bearer tokens;
- JWT values;
- database passwords;
- private keys;
- secret environment variables;
- full payment credentials;

- sensitive authentication material;
- other reusable secrets.

Sensitive information must not be exposed merely because the Audit Log is privileged.

23. AI SCAN DATA MINIMISATION

Full AI Scan reports, internal findings, confidential recommendations and assessment content must not be unnecessarily duplicated into Audit Log entries.

Prefer:

Elite Scan 872 - Report Version 4 - Approved

rather than copying the contents of Report Version 4 into the Audit Log.

Audit evidence should reference the relevant resource/version.

The underlying authorised resource remains the location for substantive content.

24. PERSONAL DATA MINIMISATION

Audit records must contain only the personal information reasonably necessary for:

- security;
- accountability;
- governance;
- investigation;
- operational traceability;
- legal/compliance purposes where applicable.

The Audit Log must not become an unrestricted secondary data warehouse.

25. AUDIT LOG ACCESS CONTROL

Audit Log access is a separate privileged capability.

Platform role alone must not automatically grant unrestricted Audit Log access.

Super Admin

May have full authorised Audit Log visibility.

Admin

No general Audit Log access merely because the user is Admin.

Developer

No general Audit Log access merely because the user is Developer.

Technical Operator

No general Audit Log access merely because the user is Technical Operator.

Client

No access to IO SKY's internal Audit Log.

If Client-facing activity history is later required, that should be designed as a separate authorised Client-facing capability rather than exposing the internal governance Audit Log.

26. GRANULAR AUDIT PERMISSIONS

The technical permission model must support appropriately granular audit capabilities.

The exact permission naming convention remains an implementation decision, but conceptually the platform should distinguish capabilities such as:

- Audit Log View;
- Security Audit View;
- Governance Audit View;
- Audit Export;
- Audit Integrity Status;
- other scoped Audit capabilities where required.

For illustration only:

audit.view

audit.security.view

```
audit.governance.view
audit.export
```

These names are not mandatory technical identifiers.

The principle is mandatory:

Audit access must be explicitly authorised.

27. AUDIT LOG SEARCH & INVESTIGATION UX

The Audit Log must be designed for practical investigation.

A user must not be required to scroll through thousands of records to locate one event.

The Audit Log must provide:

- search;
- filtering;
- combined filters;
- sorting;
- efficient result loading;
- clear active-filter visibility;
- reset/clear filters;
- individual record detail view.

Search and filtering must remain practical as the volume of Audit records grows.

28. GLOBAL SEARCH

The Audit Log must provide a search capability capable of locating relevant records using supported identifiers/context such as:

- user;
- actor;
- resource ID;
- Client;
- Project;
- AI Scan;
- Event/Action Type;
- other indexed safe audit metadata.

Search behavior should be designed to avoid unnecessary exposure of unauthorised information.

Search must operate only within the current user's authorised Audit Log scope.

29. DATE AND TIME FILTERING

Users with Audit Log access must be able to filter by time period.

At minimum:

- Today;
- Last 7 Days;
- Last 30 Days;
- Custom Date Range.

Custom Date Range should support:

```
From -> To
```

Where useful, time-of-day filtering may additionally be supported.

Dates displayed in the UI must follow the approved timezone behavior while the underlying timestamp remains authoritative.

30. ACTOR FILTER

The Audit Log must support filtering by:

- specific human user;
- System Actor;
- Service Actor.

This makes it possible to answer:

"Show me everything this specific account did during this period."

31. ROLE FILTER

Audit Log filtering must support platform roles including:

- Super Admin;
- Admin;
- Developer;
- Technical Operator;
- Client where relevant to audited platform activity;
- System/Service.

This allows investigations such as:

Role = Developer

combined with:

Environment = Production

and:

Outcome = DENIED

32. ACTION / EVENT TYPE FILTER

The user must be able to filter by Action/Event Type.

Examples:

- Login;
- Access Denied;
- Permission Granted;
- Permission Revoked;
- MFA Changed;
- Project Assigned;
- Reviewer Assigned;
- AI Scan Approved;
- AI Scan Published;
- AI Scan Publication Failed;
- Security Action;
- Audit Export;
- Credential Rotated.

The actual list should derive from the implemented stable audit event registry.

33. CATEGORY FILTER

Audit records should be classifiable into logical categories such as:

- Authentication;
- Access;
- Permissions;
- Security;
- AI Scan;
- Client / Account;
- Project;
- Billing;
- Platform;
- Configuration;
- Infrastructure;
- Governance;
- Audit Integrity.

The category system should assist investigation without replacing the stable underlying Event/Action Type.

34. RESOURCE TYPE FILTER

The Audit Log must support filtering by relevant resource type.

Examples:

- User;
- Client;

- Account;
- Project;
- AI Scan;
- Discovery Call;
- Invoice;
- Integration;
- Security Incident;
- Configuration;
- Environment.

35. SPECIFIC RESOURCE FILTER

A user must be able to isolate the history of one specific authorised resource.

Example:

```
Resource Type = AI Scan
Resource ID = SCAN-872
```

The result should show only authorised Audit records associated with that Scan.

This is essential for incident reconstruction and operational review.

36. CLIENT / ORGANISATION FILTER

Authorised users must be able to filter Audit records by:

- Client;
- organisation;
- tenant.

This must never weaken tenant isolation.

The fact that a filter contains another Client's name must not make that Client's records visible to a user who lacks access.

37. PROJECT FILTER

Where relevant, authorised users must be able to filter by a specific Project.

Example:

```
Client = Client A
Project = Project X
Role = Developer
Date = Last 30 Days
```

This should return only matching authorised Audit records.

38. AI SCAN FILTER

The Audit Log must support AI Scan-specific investigation.

Users should be able to filter by:

- Scan;
- Scan type where relevant;
- Scan lifecycle action;
- Reviewer;
- Publisher;
- approval;
- publication;
- publication failure.

Example:

```
Client = Client A
Resource = Elite Scan #872
Action = Approve / Publish
```

This should make the entire approval/publication accountability chain easy to reconstruct.

39. ENVIRONMENT FILTER

Where environment is relevant, the Audit Log must support:

- Development;
- Staging;
- Production.

Example:

```
Environment = Production
Role = Developer
Outcome = DENIED
```

This enables focused security investigation.

Environment filtering must still respect the user's underlying environment authorization.

40. OUTCOME FILTER

Users must be able to filter by outcome, at minimum:

- Success;
- Failed;
- Denied.

Where technically useful, additional outcomes may be supported.

41. SEVERITY / SIGNIFICANCE FILTER

Where audited events have a meaningful security/operational severity, the Audit Log should support filtering by significance/severity.

Examples may include:

- Critical;
- High;
- Normal;
- Informational.

This must not require every ordinary Audit record to be artificially assigned a security severity.

42. HUMAN VERSUS SYSTEM FILTER

Users must be able to distinguish:

- Human actions;
- System actions;
- Service actions.

This is important during incident investigation.

Example:

"Did the Admin publish this Scan, or did an automated process perform the action?"

The Audit Log should answer this clearly.

43. COMBINED FILTERING

Filters must be combinable.

This is a hard usability requirement.

Example investigation:

```
Environment = Production
Role = Developer
Category = Security
Outcome = Denied
Date = 01 Aug - 31 Aug
```

Another example:

```
Client = Client A
Resource Type = AI Scan
Action = Approved / Published
```

Date = Last 30 Days

The platform must return only records matching the authorised combined query.

44. ACTIVE FILTER VISIBILITY

When filters are active, the UI must clearly show them.

A user should immediately understand why only certain records are displayed.

Example:

Active filters

Production Developer Denied Security Last 30 Days

45. CLEAR FILTERS

The Audit Log must provide an obvious:

Clear All Filters

or equivalent function.

A user must not need to manually reset every filter individually.

46. SORTING

At minimum, users must be able to sort:

- newest first;
- oldest first.

Additional useful sorting may be added where appropriate.

The default should normally prioritise the most recent relevant Audit records.

47. LARGE-DATASET HANDLING

The Audit Log may eventually contain very large numbers of records.

The interface must therefore not depend on loading the entire history into the browser.

The system must use an appropriate scalable result-loading approach such as:

- pagination;
- cursor-based pagination;
- incremental loading;
- or another technically suitable method.

The exact implementation is the development team's responsibility.

The functional requirement is:

Large Audit Log datasets must remain practically navigable and must not require loading or scrolling through the entire history.

48. AUDIT LOG PERFORMANCE

Search, filtering, sorting and opening Audit details must remain operationally usable as Audit Log volume grows.

This specification does not prescribe a specific database index design.

Developers remain responsible for:

- indexing;
- query architecture;
- partitioning;
- caching where appropriate;
- archival architecture;
- pagination;
- storage design;
- query optimisation.

However:

A technically correct Audit Log that becomes impractical to search at realistic platform scale is not considered complete.

Performance must therefore be tested using representative high-volume Audit data before Production readiness.

49. RESOURCE-LEVEL "VIEW AUDIT HISTORY"

Where useful, authorised platform resources should provide direct access to their filtered Audit history.

Examples:

Client

View Audit History

opens:

Audit Log -> Client = Current Client

Project

View Audit History

opens:

Audit Log -> Project = Current Project

AI Scan

View Audit History

opens:

Audit Log -> AI Scan = Current Scan

User

View Audit History

opens:

Audit Log -> Actor / User = Current User

This does not create additional access.

The destination must still apply the normal Audit Log authorization rules.

If the user is not authorised to view Audit records, the button/function must not grant access.

50. AUDIT RECORD DETAIL VIEW

Selecting an Audit record must open a clear structured detail view.

The primary UI must not simply expose a large raw JSON payload.

The detail view should clearly present, where applicable:

Event

- Event / Action Type;
- Category;
- Outcome;
- severity/significance where relevant.

Actor

- Actor name/identifier;
- Actor Type;
- Role;
- relevant permission/scope context.

Time

- authoritative timestamp.

Resource

- Resource Type;
- Resource ID;
- Client/Organisation;
- Project;
- AI Scan;
- Environment.

Change

- relevant previous state;
- relevant resulting state.

Context

- reason/justification where applicable;
- safe correlation/request reference;
- other authorised investigation context.

Integrity

- integrity status or verification context where appropriate.

Raw technical metadata may be available to specifically authorised technical/security users where useful, but should not replace the structured primary experience.

51. NAVIGATION FROM AUDIT RECORD TO RESOURCE

Where the underlying resource still exists and the user is authorised to access it, the Audit detail view may provide a secure link to that resource.

Examples:

- Open AI Scan;
- Open Project;
- Open Client;
- Open Security Incident;
- Open User.

The Audit Log link must not bypass normal resource authorization.

52. AUDIT EXPORT

Audit records must be exportable where legitimate investigation, security, governance, legal or compliance needs require it.

Export must be separately authorised.

Possession of the Admin role alone must not automatically grant unrestricted export rights.

Export must respect:

- Audit permission;
- Client scope;
- Project scope;
- resource scope;
- environment scope;
- security scope;
- active filters where applicable.

53. FILTERED EXPORT

Where export is permitted, users should be able to export the relevant filtered result set rather than being forced to export the entire Audit database.

Example:

Client A + AI Scan + 1-31 August + Publish Events

The export should contain only the authorised records matching that scope.

54. EXPORT TRACEABILITY

Material Audit Log exports must themselves be auditable where appropriate.

The Audit Log should be capable of recording:

- who initiated the export;
- when;
- relevant scope/filter context;
- export type;
- outcome.

The Audit Log must not store the exported sensitive data again merely to prove that an export occurred.

55. AUDIT DATA RETENTION

Audit retention must be policy-driven.

The platform architecture must support retention rules appropriate to different categories of Audit data where required.

Exact Production retention periods are not fixed in this specification. Retention behavior must be configurable so the approved IO SKY retention policy can be applied without architectural redesign.

Retention must support requirements arising from:

- applicable law;
- privacy requirements;
- contractual requirements;
- security requirements;
- governance requirements;
- IO SKY's approved data-retention policy.

Retention must therefore not be permanently hard-coded around an arbitrary single duration without approved business/legal justification.

56. RETENTION EXPIRY VERSUS MANUAL DELETION

Policy-driven retention expiry is fundamentally different from a user manually deleting an inconvenient Audit record.

The platform must not provide ordinary users with a capability such as:

Delete this Audit Record

When an Audit record legitimately reaches its approved retention limit, any removal/archival process must be controlled according to the approved retention architecture.

The development team must design this so that retention cannot be abused as a hidden manual deletion mechanism.

57. LEGAL HOLD / RETENTION OVERRIDE CAPABILITY

The architecture must support controlled preservation of relevant Audit evidence when ordinary retention would otherwise remove it but authorised preservation is required for:

- security investigation;
- dispute;
- legal requirement;
- incident;
- compliance matter.

The exact legal-hold authority and policy will be supplied through the approved IO SKY legal/privacy framework. The technical design must support such a controlled capability and must not make it impossible to apply.

58. TENANT ISOLATION

Audit Log access must enforce Client/tenant isolation.

Examples:

- Client Manager assigned only to Client A must not retrieve Client B audit evidence through search.
- A Project-specific authorised user must not expand Audit scope by changing URL parameters.
- Search suggestions must not leak unauthorised Client/resource names.
- Export must not include records outside authorised scope.
- Resource-level Audit History must not bypass tenant restrictions.

Backend authorization remains authoritative.

59. ENVIRONMENT ISOLATION

Audit evidence must preserve the distinction between:

- Development;
- Staging;
- Production.

Environment-sensitive Audit records must clearly identify their environment.

A user authorised only for Staging must not gain access to Production Audit evidence merely because they have general technical access.

Production Audit access should be treated according to Production authorization.

60. SEARCH AND FILTER SECURITY

Search and filtering are not authorization mechanisms.

A query such as:

```
Client = Client B
```

must not return Client B data if the current user lacks Client B Audit access.

This applies to:

- search results;
- filter values;
- autocomplete;
- counts;
- exports;
- URLs;
- APIs;
- deep links;
- record details.

The backend must apply authorization before returning results.

61. PAGINATION SECURITY

Pagination/incremental loading must preserve authorization.

A user must not be able to:

- manipulate cursors;
- modify page parameters;
- alter offsets;
- change resource identifiers

to retrieve unauthorised Audit records.

62. AUDIT LOG ACCESS MUST ITSELF BE PROTECTED

The Audit Log is a sensitive platform resource.

Appropriate protections must include:

- authenticated access;
- backend authorization;
- tenant isolation;
- environment isolation;
- privileged permission enforcement;
- secure exports;
- session/security controls consistent with the role.

Sensitive Audit access must not rely solely on hidden frontend navigation.

63. AUDIT LOG VIEWING / EXPORT ACCOUNTABILITY

Not every ordinary Audit Log page view needs to generate another Audit record, otherwise the system can create recursive noise.

However, sensitive operations such as:

- privileged Audit export;
- sensitive Audit integrity administration;
- changes to Audit configuration;
- changes to Audit retention;
- integrity verification actions where governance-significant;
- other high-risk Audit administration

must themselves be traceable where appropriate.

64. AUDIT INTEGRITY STATUS

The system must provide an appropriate mechanism for authorised security/governance users to determine whether Audit integrity controls are healthy.

The exact UI is a development decision.

However, the system must not silently discover an integrity failure and leave it invisible.

At minimum, authorised users must be able to determine:

- integrity healthy;
- integrity verification failed;
- verification unavailable/unknown where applicable;
- investigation required.

65. AUDIT INTEGRITY FAILURE

When the platform detects or has strong evidence of an Audit Log integrity problem, it must create the corresponding security/governance workflow using the required event:

`AUDIT_INTEGRITY_ACTION_REQUIRED`

This event must be aligned with and included in the Notification Event Catalogue if it is not already present.

The event must route to:

- Super Admin;
- authorised Security/Governance users as applicable.

Priority:

- P1 where evidence suggests serious active compromise or loss of Audit integrity;
- P2 where material investigation is required but immediate critical compromise is not established.

P1 may bypass quiet hours.

This event must not be digest-delayed when immediate investigation is required.

66. AUDIT INTEGRITY EVENT MUST ITSELF BE TRACEABLE

Detection, acknowledgement, investigation and resolution of Audit integrity incidents must themselves be traceable through an appropriate protected mechanism.

The implementation must avoid a circular design where compromise of the same store destroys all evidence that compromise was detected.

The development team must explain how the chosen architecture handles this risk.

67. BACKUPS, RESTORE AND AUDIT INTEGRITY

Backup and restore architecture must not silently destroy or invalidate Audit integrity guarantees.

The development team must document how:

- Audit records are backed up;
- integrity evidence is preserved;
- restore affects integrity verification;
- restored Audit history is verified;
- retention remains consistent after restore.

A successful database restore that silently makes Audit evidence unverifiable is not an acceptable Production result.

68. CLOCK AND TIMESTAMP INTEGRITY

Audit timestamps must originate from a reliable server-side source.

The platform must not rely on the user's local device clock for authoritative Audit time.

Where multiple services create Audit records, the technical architecture must provide sufficiently consistent time handling to support meaningful event reconstruction.

Display may use user/local timezone.

Stored evidence must preserve an authoritative canonical time reference.

69. CORRELATION

Where useful for investigation, related Audit records should support correlation.

Examples:

- one request causes multiple backend actions;
- one AI Scan Publish operation touches multiple resources;
- one privileged access change creates several related records;
- one security incident involves several system actions.

A safe correlation identifier may be used.

Correlation identifiers must not themselves contain secrets.

70. AUDIT EVENT REGISTRY

Audited actions should use stable, defined action/event identifiers.

Human-readable wording may change or be translated.

The stable underlying identifier must remain consistent.

Example:

```
AI_SCAN_REVIEW_APPROVED
```

may display as:

"Elite Scan approved by Reviewer"

but the stable audit event type remains unchanged.

This supports:

- filtering;
- analytics;
- investigations;
- automated testing;
- localisation;
- long-term historical consistency.

71. AUDIT EVENT CATEGORIES

Each stable Audit Event Type should belong to an appropriate logical category.

Recommended categories include:

- AUTHENTICATION;
- ACCOUNT_SECURITY;
- ACCESS_CONTROL;
- PERMISSIONS;
- CLIENT_ACCOUNT;
- PROJECT;
- AI_SCAN;
- BILLING;
- SECURITY;
- PLATFORM;
- INTEGRATION;
- CONFIGURATION;
- INFRASTRUCTURE;
- GOVERNANCE;
- AUDIT_INTEGRITY.

Developers may refine the internal taxonomy provided it remains clear and supports the required UI filtering.

72. LOCALISATION

Audit event semantics must not depend on a single hard-coded display language.

Stable Audit Event IDs should remain language-independent.

Human-readable UI labels may use the platform's localisation architecture.

Changing display language must not change:

- event identity;
- stored outcome;
- actor;
- resource;
- timestamp;
- authorization;
- historical meaning.

73. DEFAULT AUDIT LOG VIEW

The default Audit Log view should be calm, structured and investigation-oriented.

Recommended primary columns:

- Timestamp;
- Actor;
- Role / Actor Type;
- Action;
- Category;
- Resource;
- Client / Organisation;
- Environment;
- Outcome.

The exact responsive UI can be determined by the development team.

Not every available field needs to appear as a permanent table column.

Additional information belongs in the Audit detail view.

74. NO RAW-LOG UX AS PRIMARY INTERFACE

IO SKY's Audit Log must not simply expose:

- unstructured server logs;
- raw JSON streams;
- terminal output;
- database rows without interpretation.

Technical raw data may exist underneath the system and may be available to appropriately authorised technical personnel where necessary.

The primary governance interface must remain understandable and usable.

75. EMPTY STATE

When no records match the current authorised query, the UI should clearly state that no Audit records match the current filters/search.

It should not appear broken.

Example:

No audit records match the current filters.

The user should be able to clear/reset filters easily.

76. ERROR STATE

If Audit records cannot be retrieved, the platform must distinguish:

- no matching records;
- permission denied;
- retrieval/system failure.

These states must not be misleadingly presented as equivalent.

Security-sensitive implementation details should not be exposed in user-facing errors.

77. AUDIT LOG AVAILABILITY

The Audit Log is a governance/security capability and must be treated as operationally important.

A temporary Audit UI failure must not silently cause security-critical actions to stop being recorded if the backend Audit subsystem remains capable of recording them.

The recording mechanism and viewing interface should therefore not be unnecessarily coupled.

The exact technical architecture remains a developer decision.

78. FAILURE TO WRITE REQUIRED AUDIT EVIDENCE

For actions defined as requiring mandatory Audit evidence, the system must not silently proceed as though auditing succeeded when required evidence could not be safely recorded.

The development team must define and document the failure behavior appropriate to each class of privileged action.

For especially sensitive actions, fail-closed behavior should be considered where technically and operationally appropriate.

However, this specification does not mandate that every ordinary business action must fail solely because a non-critical Audit subsystem is temporarily degraded.

The developers must propose a safe policy distinguishing:

- mandatory security/privileged Audit writes;
- important business Audit writes;
- non-critical operational evidence.

79. MONITORING OF AUDIT PIPELINE HEALTH

The technical implementation must make Audit subsystem health observable.

Developers/operators should be able to determine whether:

- required Audit events are being accepted;
- records are being persisted;
- integrity controls are functioning;
- errors are occurring;
- queues/backlogs exist where relevant;
- delivery/storage has materially failed.

Routine healthy Audit processing should not create Notification Center noise.

Material Audit failure may require a Platform/Security/Governance event.

80. AUDIT LOG AND DATABASE ADMINISTRATION

The development team must explicitly document which infrastructure/database-level identities can technically:

- write Audit records;
- alter storage;
- delete storage;
- change retention;
- modify integrity mechanisms;
- restore backups.

Least-privilege principles should be applied.

Ordinary application users must not receive database-level Audit modification authority.

81. DIRECT DATABASE MODIFICATION RISK

Because application-level immutability alone does not protect against a sufficiently privileged database actor, the chosen integrity architecture must address or detect direct underlying data manipulation.

This is a central reason for requiring tamper evidence.

The development team must demonstrate that simply bypassing the application UI does not silently defeat the Audit integrity model.

82. AUDIT EXPORT SECURITY

Audit exports must:

- require backend authorization;
- respect active scope;
- exclude secrets;
- preserve tenant isolation;
- preserve environment isolation;
- use safe file delivery;
- avoid public/unprotected download URLs;
- expire temporary access where applicable.

The development team determines the secure export mechanism.

83. AUDIT EXPORT CONTENT

Where appropriate, exported records should preserve useful investigation fields such as:

- Audit Record ID;
- timestamp;
- actor;
- actor type;
- role;
- event/action type;
- category;
- resource;
- Client/organisation;
- Project;
- environment;
- outcome;
- relevant safe change information.

Sensitive data minimisation requirements remain applicable to exports.

84. AUDIT RETENTION CONFIGURATION

Where retention configuration is exposed through the platform, changing it must itself be:

- privileged;
- separately authorised;
- auditable;
- protected against accidental or malicious mass deletion.

Super Admin authority alone should not imply that arbitrary historical evidence can instantly be destroyed without controlled policy.

The governance workflow must remain configurable so the approved IO SKY retention policy can be applied. Developers must not hard-code or independently determine the final legal retention governance rules.

85. PRIVACY / DATA SUBJECT OPERATIONS

Audit evidence may contain personal information necessary for legitimate security/accountability purposes.

Privacy operations must therefore not blindly delete Audit evidence merely because an associated user record changes or is removed.

The final implementation must be capable of reconciling:

- privacy requirements;
- data minimisation;
- retention obligations;
- security/accountability needs.

The implementation must remain configurable to the approved IO SKY privacy and retention rules. Developers must not independently invent legal retention or deletion periods.

86. USER DELETION / DEACTIVATION

If a user is deactivated or legitimately removed, historical Audit evidence of actions previously performed by that user must remain attributable in a safe way for the applicable retention period.

The Audit Log must not become historically meaningless because an account no longer exists.

The implementation may preserve an immutable actor reference and safe display metadata according to the approved privacy architecture.

87. RESOURCE DELETION

Deleting or archiving a Project, Scan or other resource must not automatically destroy the historical Audit evidence related to that resource.

The Audit Log should retain sufficient safe reference information to understand what the historical action concerned for the applicable retention period.

88. INVESTIGATION WORKFLOW EXAMPLES

The following investigations must be practical without manually scrolling through large Audit histories.

Example A - Developer Production Access

Filter:

Environment = Production
Role = Developer
Category = Access/Security
Outcome = Denied
Date = Last 30 Days

Result:

Only matching authorised Production access events.

Example B - AI Scan Publication

Filter:

Client = Client A
Resource Type = AI Scan
Scan = Elite Scan #872
Action = Review Approved / Publish

Result should allow an authorised investigator to reconstruct:

1. who reviewed;
2. when;
3. which report version;
4. who approved;
5. when;
6. who published;
7. which version;
8. in which language;
9. whether publication succeeded;
10. whether any failed publication occurred first.

Example C - Permission Investigation

Filter:

Actor = Super Admin X
Category = Permissions
Date = 01 August - 31 August

Result:

All authorised relevant permission/scope changes performed by that actor.

Example D - Client Investigation

From:

Client A -> View Audit History

The Audit Log opens pre-filtered:

Client = Client A

Additional filters can then be applied.

Example E - Security Incident

Filter:

Category = Security
Severity = Critical
Environment = Production
Date = Last 7 Days

Result:

Relevant authorised critical Production security Audit evidence.

89. SECURITY REQUIREMENTS SUMMARY

The Audit Log must prevent:

- cross-Client leakage;
- cross-Project leakage;
- cross-environment leakage;

- frontend-only authorization;
- unrestricted Admin access;
- unrestricted Developer access;
- unrestricted Technical Operator access;
- secret exposure;
- token exposure;
- ordinary manual deletion;
- ordinary record editing;
- hidden history rewriting;
- unauthorized exports;
- URL/filter manipulation bypass;
- stale access bypass;
- unauthorised resource deep links.

90. AUDIT INTEGRITY ACCEPTANCE TESTS

Before Production readiness, QA must verify at minimum:

1. Client cannot access internal IO SKY Audit Log.
2. Admin without Audit permission cannot access Audit Log.
3. Developer without Audit permission cannot access Audit Log.
4. Technical Operator without Audit permission cannot access Audit Log.
5. Super Admin can access authorised Audit Log functionality.
6. Audit records cannot be edited through normal UI.
7. Audit records cannot be individually deleted through normal UI.
8. Admin cannot alter historical Audit evidence.
9. Super Admin cannot alter/delete individual historical Audit evidence through ordinary platform functionality.
10. Permission Granted and Permission Revoked remain separate records.
11. Role changes are traceable.
12. Permission changes are traceable.
13. Scope changes are traceable.
14. Privileged access changes are traceable.
15. MFA/security changes are traceable where required.
16. Security Center privileged actions are traceable.
17. AI Scan Reviewer assignment is traceable.
18. AI Scan Review Approval is traceable.
19. AI Scan Publish is separately traceable.
20. Failed AI Scan Publish remains traceable after later successful Publish.
21. Correct report version can be identified from publication Audit evidence.
22. Relevant Client/organisation can be identified.
23. Environment can be identified where applicable.
24. Human actor and System/Service actor can be distinguished.
25. Relevant failed privileged actions are traceable.
26. Relevant denied privileged access is traceable.
27. Passwords never appear in Audit records.
28. API keys never appear.
29. Access/refresh tokens never appear.
30. Database passwords never appear.
31. MFA secrets never appear.
32. Private keys never appear.
33. Audit search works.
34. Date filtering works.
35. Actor filtering works.
36. Role filtering works.
37. Action/Event filtering works.
38. Category filtering works.
39. Resource filtering works.
40. Client/organisation filtering works.
41. Project filtering works.
42. AI Scan filtering works.
43. Environment filtering works.
44. Outcome filtering works.

45. Human/System filtering works.
46. Multiple filters can be combined.
47. Clear All Filters works.
48. Newest/oldest sorting works.
49. Large datasets do not require loading all records at once.
50. View Audit History from an authorised Client opens correctly scoped results.
51. View Audit History from an authorised Project opens correctly scoped results.
52. View Audit History from an authorised AI Scan opens correctly scoped results.
53. View Audit History from an authorised User opens correctly scoped results.
54. Resource-level Audit History cannot bypass Audit permissions.
55. Search does not expose unauthorised Client names/data.
56. Filter options do not leak unauthorised resources.
57. Pagination/cursor manipulation cannot bypass authorization.
58. Export requires appropriate permission.
59. Export respects active scope.
60. Export does not contain secrets.
61. Relevant Audit export is itself traceable.
62. Staging-only user cannot retrieve Production Audit records.
63. Client A-scoped user cannot retrieve Client B Audit records.
64. Deleted/deactivated user does not make historical evidence meaningless.
65. Deleted/archived resource does not automatically erase historical Audit evidence.
66. Authoritative timestamp comes from server-side source.
67. Reprocessing/retrying an Audit event does not create inappropriate duplicate evidence.
68. Chosen tamper-evidence mechanism detects the manipulation scenarios it claims to detect.
69. Integrity verification failure becomes observable.
70. Material integrity failure can generate AUDIT_INTEGRITY_ACTION_REQUIRED.
71. P1 Audit integrity event can follow critical notification behavior.
72. Backup/restore preserves or correctly re-establishes Audit integrity verification.
73. Retention processing cannot be used as ordinary manual deletion.
74. Retention configuration changes are privileged and traceable where implemented.
75. Required Audit pipeline failure is observable.
76. Privileged action behavior during mandatory Audit-write failure follows documented safe policy.
77. Audit Log detail view presents structured human-readable information.
78. Raw JSON is not the only primary investigation interface.
79. Deep links from Audit records re-check current resource authorization.
80. Audit search/filter performance remains practically usable against representative high-volume data.

91. REQUIRED SECURITY TESTING

The development team must test the Audit Log specifically against attempts to:

- alter an Audit record;
- delete an Audit record;
- bypass UI restrictions;
- directly manipulate protected underlying data in an approved security test;
- retrieve another Client's records;
- retrieve another Project's records;
- retrieve Production records from Staging-only authorization;
- manipulate filter/API parameters;
- manipulate pagination/cursors;
- export outside authorised scope;
- expose secrets through before/after state;
- bypass revoked permissions through an old URL;
- access an underlying resource through an Audit deep link after resource access is revoked.

Test results must be documented.

92. TAMPER-EVIDENCE DEMONSTRATION

Before the Audit Log can be considered Production-ready, the development team must demonstrate the chosen integrity mechanism.

At minimum, IO SKY must receive evidence showing:

1. normal Audit records being created;

2. application users cannot modify them;
 3. application users cannot individually delete them;
 4. a controlled tampering test;
 5. the integrity mechanism detecting the tested tampering;
 6. resulting system behavior;
 7. resulting security/governance alert behavior where applicable;
 8. successful verification after legitimate backup/restore or recovery procedure where applicable.
- A statement that the Audit Log is "immutable" is not sufficient without evidence.

93. PERFORMANCE TESTING

The development team must test Audit Log retrieval against representative high-volume datasets.

The purpose is to demonstrate that:

- default loading remains practical;
- search remains practical;
- combined filters remain practical;
- sorting remains practical;
- pagination remains stable;
- exports remain controlled;
- authorization remains correct under load.

The test volume should reasonably represent expected Production growth with additional headroom.

94. AUDIT LOG OBSERVABILITY

The development team must provide sufficient technical observability to diagnose:

- failed Audit writes;
- Audit processing backlog;
- integrity verification failure;
- storage failure;
- excessive query latency;
- export failures;
- other material Audit subsystem failures.

This technical observability does not mean every healthy metric becomes a Notification Center event.

95. DEFINITION OF DONE

The Audit Log work is not complete merely because a database table named `audit_log` exists.

It is complete only when:

- required event categories are captured;
- privileged actions are traceable;
- records are application-immutable;
- tamper evidence is implemented;
- integrity failure is detectable;
- server-side authoritative auditing is used;
- access is separately authorised;
- tenant isolation works;
- Project/resource scoping works;
- environment isolation works;
- secrets are excluded;
- human/system actors are distinguishable;
- failed/denied privileged actions are handled appropriately;
- search works;
- all required filters work;
- combined filtering works;
- sorting works;
- large datasets are navigable;
- resource-level View Audit History works;
- structured detail view works;
- secure export works;
- export authorization works;

- relevant export actions are traceable;
- retention architecture is policy-driven;
- backup/restore does not silently destroy integrity;
- integrity status is observable;
- Audit subsystem failure is observable;
- Notification Event Catalogue integration works for Audit integrity incidents;
- acceptance tests pass;
- security testing passes;
- tamper-evidence demonstration is completed;
- high-volume performance is validated;
- implementation documentation is delivered.

96. REQUIRED DEVELOPER DOCUMENTATION

At handoff, the development team must document:

Architecture

- Audit event generation architecture;
- storage architecture;
- integrity architecture;
- authorization model;
- tenant/environment scoping;
- retention architecture;
- backup/restore interaction;
- search/query architecture;
- export architecture.

Security

- actors capable of writing Audit records;
- actors capable of modifying underlying storage;
- protections against unauthorised alteration;
- integrity verification mechanism;
- integrity failure handling;
- secret/data minimisation controls.

Operations

- monitoring;
- failure handling;
- Audit-write failure policy;
- recovery;
- backup/restore verification;
- performance characteristics.

QA

- acceptance test results;
- security test results;
- tenant isolation results;
- environment isolation results;
- tamper test;
- high-volume search/filter test.

97. IMPLEMENTATION DECISIONS LEFT TO DEVELOPMENT TEAM

The following implementation choices remain the responsibility of the development team:

- database technology;
- table structure;
- event bus;
- queueing;
- workers;
- hash algorithm;
- cryptographic chaining;

- signing mechanism;
- immutable storage provider;
- indexing architecture;
- partitioning;
- caching;
- pagination implementation;
- export format/provider;
- archival storage;
- observability stack;
- backup mechanism;
- correlation implementation.

These are technical engineering decisions.

However, the chosen architecture must satisfy all functional, security, governance, usability and acceptance requirements in this specification.

98. MANDATORY FUNCTIONAL REQUIREMENTS - NON-NEGOTIABLE

The following are functional requirements, not optional technical suggestions:

1. Audit Log and Notification Center remain separate.
2. Audit records are append-only from the application perspective.
3. Historical records cannot be ordinarily edited.
4. Historical records cannot be individually deleted through normal platform functionality.
5. Super Admin does not receive an ordinary Audit Edit/Delete capability.
6. Technical tamper detection is required.
7. Material privileged actions are auditable.
8. AI Scan Review Approval and Publish remain separately traceable.
9. Secrets must not be stored in Audit records.
10. Audit access is separately authorised.
11. Platform roles alone do not automatically grant unrestricted Audit access.
12. Tenant isolation applies to Audit data.
13. Environment isolation applies to Audit data.
14. Human and system/service actions must be distinguishable where applicable.
15. Relevant failed/denied privileged actions must be traceable.
16. Audit Log must be searchable.
17. Audit Log must support combined filters.
18. Date, Actor, Role, Event/Action, Category, Resource, Client, Project, AI Scan, Environment and Outcome filtering must be supported where applicable.
19. Large Audit histories must not require endless scrolling.
20. Resource-level View Audit History must be supported where appropriate and authorised.
21. Audit detail view must be human-readable.
22. Audit export must be separately authorised and scoped.
23. Relevant exports must themselves be traceable.
24. Retention must be policy-driven rather than based on an arbitrary hard-coded duration.
25. Integrity failure must be detectable and capable of triggering the corresponding security/governance flow.
26. Developers must demonstrate the integrity mechanism before Production readiness.
27. Audit search/filtering must remain practically usable at scale.
28. Backend authorization remains authoritative for every Audit query, filter, export and deep link.

99. RETENTION CONFIGURATION REQUIREMENT

Exact Production Audit Log retention periods are not fixed in this specification.

The technical platform must support configurable retention so that the approved IO SKY retention policy can be applied without architectural redesign. The configuration must be capable of supporting requirements arising from:

- Privacy Policy;
- Data Retention Policy;
- DPA requirements;
- security requirements;
- contractual requirements;
- applicable legal requirements.

Developers must not invent, hard-code or represent an arbitrary retention period as IO SKY's approved Production retention policy.

If a temporary technical default is required during Development/Staging, it must be documented, configurable, and clearly identified as a non-Production default.

100. FINAL REQUIREMENT

The IO SKY Audit Log must make it possible for an appropriately authorised investigator to answer:

Who or what did what, to which authorised resource, for which Client/organisation, in which environment, when, with what relevant change and outcome - and can we trust that the historical evidence has not been silently altered?

And that information must be retrievable without manually searching through an unmanageable stream of logs.

The final system must therefore combine:

- Integrity
- Immutability
- Accountability
- Authorization
- Isolation
- Data minimisation
- Searchability
- Filtering
- Investigation usability
- Scalability
- Verifiability

into one controlled Audit Log capability.

That is the functional standard for IO SKY Audit Log Integrity & Governance v1.0.